

Fast Reliability Analysis with Physics-Aware Neural Networks

A Cantilever Beam on a Nonlinear Elastic Foundation

Namit Aggarwal and Aniket Raval
Project 5

September 20, 2026

Abstract

This report trains a physics-informed neural network (PINN) to predict how a cantilever beam deflects when it rests on a nonlinear elastic foundation and carries an uncertain load. Because the foundation’s resistance grows nonlinearly with deflection, the governing equation has no algebraic solution: every “ground truth” answer must come from numerically solving a nonlinear boundary-value problem. We build that numerical solver, use it to check the trained network’s accuracy, and then use both methods to run the same Monte Carlo reliability study — drawing 20,000 random combinations of material stiffness and load, and asking what fraction of them cause the beam to deflect more than an allowable limit. The trained network matches the numerical ground truth closely (well under 1% error at typical conditions) and estimates the probability of exceeding the allowable deflection as 4.61%, against 4.32% from the numerical baseline. The key result is speed: the numerical baseline takes **24.87 seconds** to evaluate all 20,000 scenarios, while the trained network takes **5.4 milliseconds** — a speed-up of roughly **4,600 times**.

Code and repository. All code used to produce this report — configuration, training, evaluation, and the Monte Carlo study — is publicly available at: <https://github.com/officialnamit/Physics-Informed-Neural-Networks>. Section 2 gives a short walkthrough of how the files in the repository fit together, before the rest of the report goes through the details.

Contents

1	Introduction	3
2	Pipeline Overview	3
3	Problem Setup	4
3.1	The beam and its foundation	4
3.2	Treating material and load as uncertain	5
3.3	The numerical baseline	5
4	Methodology	6
4.1	Training data	6
4.2	Network and physics loss	6
4.3	Training procedure	6
4.4	Uncertainty propagation	6

5	Results	6
5.1	Training convergence	6
5.2	Validation against the numerical ground truth	7
5.3	Reliability analysis	8
5.4	Timing comparison	9
6	Discussion	10
7	Where the Speed-Up Actually Comes From	11
7.1	Counting the operations on each side	11
7.2	Where the numerical solver's time actually goes	12
7.3	The real mechanism: parallel batching, not fewer computations	12
8	Is the Accuracy Trade-off Worth It?	12
9	Improving Accuracy: Training on a Wider Distribution	13
9.1	Curve-fitting accuracy improved substantially	13
9.2	The reliability estimate did not improve	14
9.3	Directions more likely to fix the reliability estimate specifically	15
10	Further Possibilities	15
11	Conclusion	16
A	Summary of Notation	16

1 Introduction

Engineers often need to know not just how a structure behaves under one specific load, but how likely it is to fail once the material properties and the load itself are treated as uncertain, as they always are in practice. Answering that properly means solving the structural problem many times over — once for every plausible combination of material stiffness and load — which becomes slow or impractical if each individual solve is itself expensive.

This report trains a neural network to solve the underlying physics directly, so that once training is done, evaluating a new material/load combination is just a quick calculation instead of a fresh numerical solve. The network is trained using *physics-informed* learning: instead of being shown labeled examples of correct answers, it is trained to satisfy the governing differential equation and boundary conditions directly, using automatic differentiation.

We apply this to a cantilever beam — fixed at one end, free at the other — resting along its length on a foundation whose stiffness increases the more it is compressed (a common behavior of real foundation materials such as compacted soil or rubber mounts). This nonlinearity means the governing equation cannot be solved with a simple formula, so every reference answer requires a numerical solve. This makes it a fair test of whether the trained network is both accurate and faster than solving the problem numerically every time.

2 Pipeline Overview

Before going through the equations and results in detail, this section gives a plain walkthrough of how the code is organized, so the rest of the report can be read against a clear picture of what runs when. Figure 1 shows the flow, matching the file names in the repository linked on the first page.

- `main.py` runs the whole pipeline in order and is the entry point.
- `config.py` holds every physical constant and training setting (beam length, material properties, network size, number of epochs) in one place.
- `data_generation.py` generates the random training points — positions along the beam paired with sampled material stiffness and load values. No dataset is loaded from disk; everything is sampled on the fly.
- `pinn_model.py` defines the network itself (the class `BeamPINN`), and `losses.py` defines the physics loss terms it is trained against.
- `train.py` runs the training loop: first Adam, then L-BFGS, and saves the trained weights to `pinn_model.pt`.
- `evaluate.py` and `uncertainty.py` both load those trained weights and check them against `numerical_baseline.py`, which solves the same problem numerically with `scipy.integrate.solve_bvp` and acts as the ground truth throughout this report. `evaluate.py` checks accuracy on a handful of specific cases; `uncertainty.py` runs the full 20,000-sample Monte Carlo study and the timing comparison.
- Every plot and every number reported in this document is written out to the `results/` folder by this pipeline, so any figure or table below can be reproduced by cloning the repository and running `python main.py`.

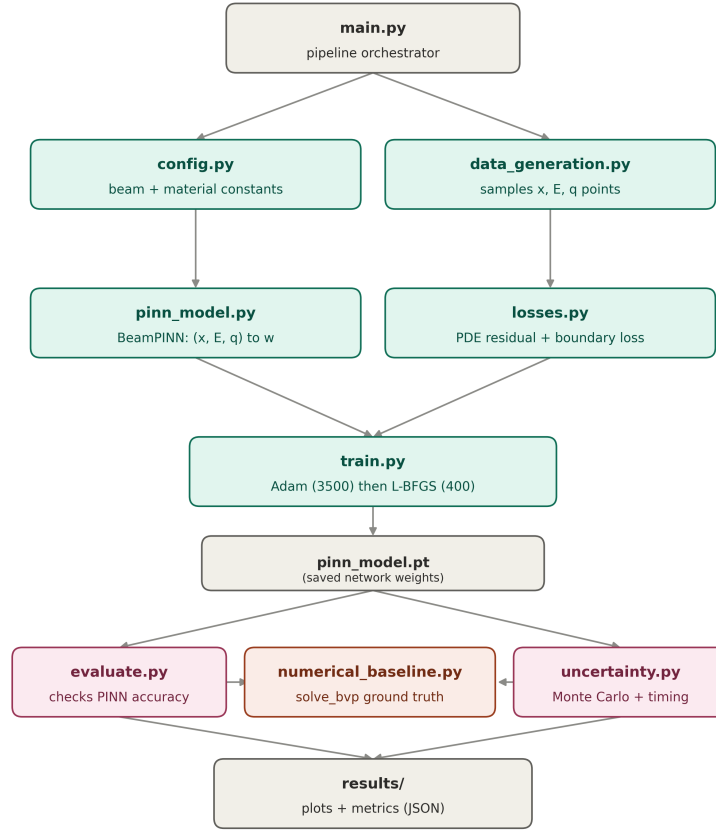


Figure 1: How the files in the repository fit together, from configuration and data generation through model training to validation and the reliability study.

3 Problem Setup

3.1 The beam and its foundation

The beam has length L , is fixed at $x = 0$, and free at $x = L$. It carries a uniformly distributed load q along its length, and rests on a foundation that pushes back against deflection with a force per unit length equal to $k_1w + k_3w^3$, where $w(x)$ is the beam's deflection at position x . The k_3w^3 term is what makes the foundation *nonlinear*: the foundation gets relatively stiffer the further it is compressed, which is a realistic property of many real supporting materials.

Combining this foundation reaction with standard beam bending theory gives the governing equation:

$$EI w''''(x) + k_1w(x) + k_3w(x)^3 = q \quad (1)$$

where E is the material's Young's modulus and I is the beam cross-section's second moment of area. The boundary conditions are the usual ones for a cantilever: at the fixed end,

$$w(0) = 0, \quad w'(0) = 0, \quad (2)$$

and at the free end, where there is no bending moment or shear force left over,

$$w''(L) = 0, \quad w'''(L) = 0. \quad (3)$$

Because of the w^3 term, Equation (1) cannot be solved by direct integration the way a simpler, linear beam equation can — there is no algebraic formula for $w(x)$. Getting an exact answer for any specific (E, q) combination requires solving this equation numerically.

3.2 Treating material and load as uncertain

Young’s modulus E and the distributed load q are treated as random rather than fixed, since in practice neither is known exactly. Table 1 lists the values used throughout this study.

Table 1: Physical setup used in this study.

Quantity	Value	Role
Beam length, L	1.0 m	fixed
Second moment of area, I	$2.0 \times 10^{-7} \text{ m}^4$	fixed
Linear foundation modulus, k_1	2000 N m^{-2}	fixed
Cubic (hardening) foundation modulus, k_3	$2.0 \times 10^9 \text{ N/m}^4$	fixed
Young’s modulus, E	nominal 200 GPa, 10% CoV	uncertain
Distributed load, q	nominal 1000 N m^{-1} , 15% CoV	uncertain
Allowable tip deflection, w_{allow}	$L/250 = 4.00 \text{ mm}$	service limit

We confirmed numerically that the nonlinear term is not a negligible detail: at higher loads, the hardening foundation noticeably reduces the deflection compared to what a purely linear foundation would predict, and this effect grows the further into the tail of the load distribution one looks — exactly the region that matters most for estimating a failure probability.

3.3 The numerical baseline

To get a trustworthy answer for any given (E, q) , we rewrite Equation (1) as a system of four first-order equations (letting $y_1 = w$, $y_2 = w'$, $y_3 = w''$, $y_4 = w'''$):

$$\frac{d}{dx} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} = \begin{pmatrix} y_2 \\ y_3 \\ y_4 \\ \frac{q - k_1 y_1 - k_3 y_1^3}{EI} \end{pmatrix}, \quad \begin{matrix} y_1(0) = 0, & y_2(0) = 0 \\ y_3(L) = 0, & y_4(L) = 0 \end{matrix} \quad (4)$$

and solve it with `scipy.integrate.solve_bvp`, a general-purpose nonlinear boundary-value-problem solver with no shortcuts specific to this problem. This is directly analogous to how a real finite-element structural analysis works: build a numerical model, iterate to convergence, and repeat entirely from scratch for every new material or load combination. Each such solve took about **1.24 milliseconds** on average in this study; a full 2D or 3D finite-element analysis of a realistic structure would typically take much longer per solve, but the same bottleneck applies — one full numerical solve required per scenario, with no way to batch or vectorize across scenarios.

4 Methodology

4.1 Training data

No external dataset is used. Instead, 1200 random collocation points (x, E, q) are drawn per training batch, with x sampled uniformly across $[0, L]$ and E, q sampled from their uncertainty distributions (Table 1). A further 150 points are drawn at each end of the beam, paired with their own random (E, q) , to enforce the boundary conditions. A fresh batch is drawn every 200 training steps. Because E and q are included as inputs alongside x , the network learns the whole family of solutions $w(x, E, q)$ across the uncertainty range, not just a single deflection curve — this is what later makes the uncertainty propagation a matter of evaluating the trained network rather than retraining it.

4.2 Network and physics loss

The network is a feedforward network with 4 hidden layers of 40 units each and tanh activations, taking (x, E, q) as input and predicting w . Automatic differentiation is used to compute w', w'', w''', w'''' with respect to x directly from the network’s output, without needing any hand-derived derivative formulas. These are combined into a physics loss enforcing Equation (1):

$$\mathcal{L}_{\text{pde}} = \frac{1}{N_{\text{pde}}} \sum_{i=1}^{N_{\text{pde}}} (E_i I w''''(x_i, E_i, q_i) + k_1 w(x_i, E_i, q_i) + k_3 w(x_i, E_i, q_i)^3 - q_i)^2 \quad (5)$$

and a boundary loss $\mathcal{L}_{\text{bc}}$ enforcing the four boundary conditions at the sampled boundary points. The total loss is $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{pde}} + 10 \mathcal{L}_{\text{bc}}$, minimized over the network’s weights.

4.3 Training procedure

Training runs in two stages: 3500 epochs of Adam (a standard stochastic gradient descent variant, with a fresh batch of points every 200 epochs), followed by 400 iterations of L-BFGS fine-tuning, a quasi-Newton method that squeezes out additional precision once Adam’s progress plateaus.

4.4 Uncertainty propagation

Once trained, we draw 20 000 independent random samples of (E, q) from their distributions and evaluate the tip deflection $w(L)$ for every sample using the trained network, in a single batched forward pass. We repeat the same 20 000 samples through the numerical baseline (Section 3.3) to get an independent comparison and a direct timing measurement. From the resulting deflection values we compute the mean, standard deviation, and the fraction of samples exceeding the allowable deflection w_{allow} — the estimated probability of failure.

5 Results

5.1 Training convergence

Figure 2 shows the total loss, physics loss, and boundary loss over the course of training. The periodic spikes correspond to each new batch of randomly resampled training points; the sharp drop at the end is the L-BFGS fine-tuning stage.

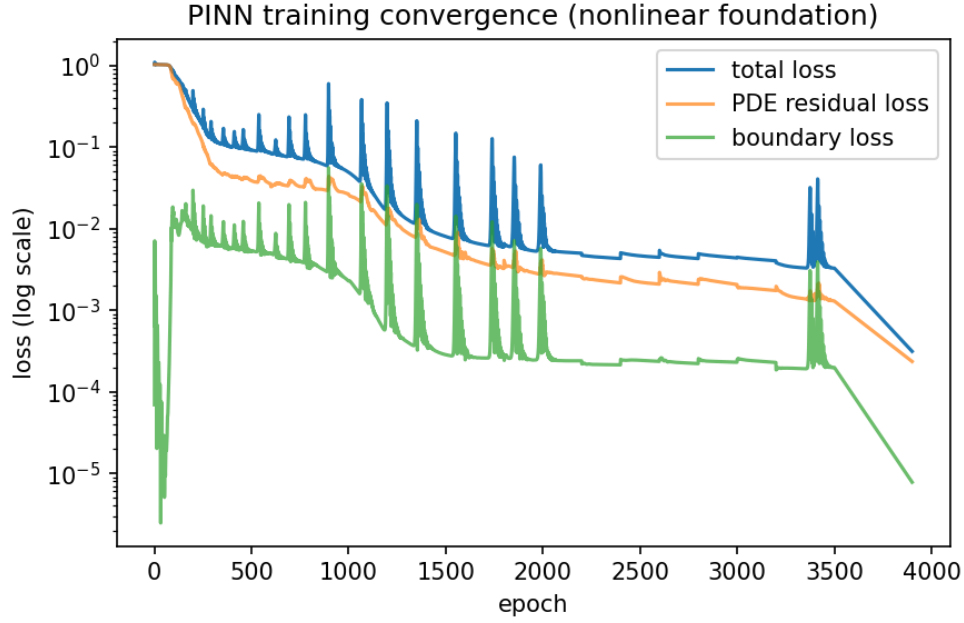


Figure 2: Training loss versus epoch (log scale).

Total training time: **135.1 seconds**.

5.2 Validation against the numerical ground truth

Figure 3 compares the trained network's predicted deflection curve against the numerical BVP solution for five material/load combinations, including a deliberately extreme case (double the nominal load) to test the network outside its training range.

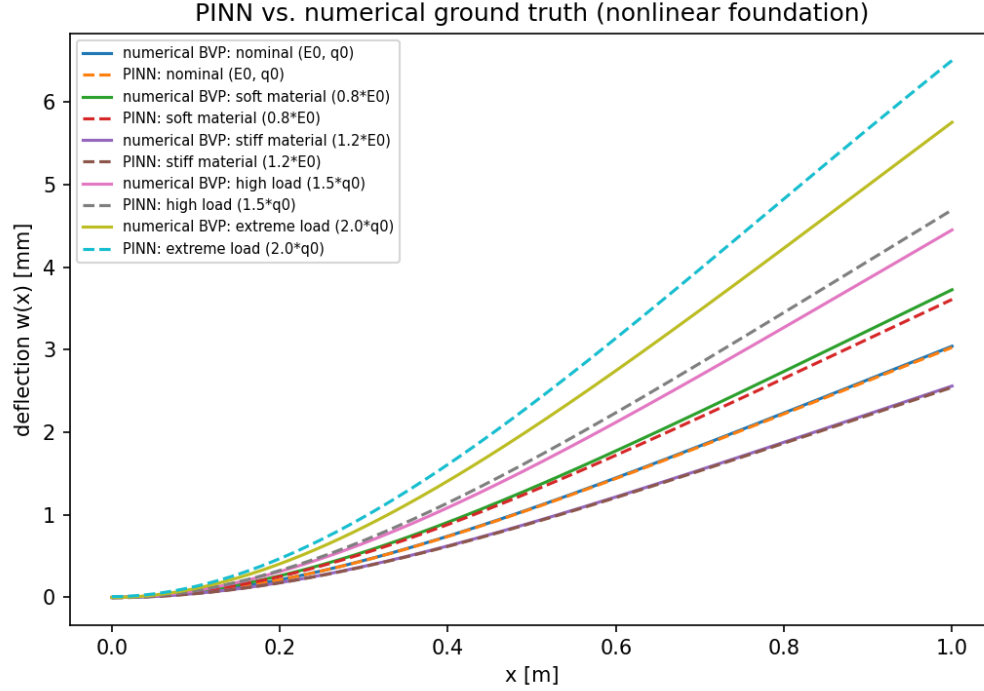


Figure 3: Predicted deflection curves from the trained network (dashed) versus the numerical ground truth (solid), for five material/load combinations.

Table 2: Relative error between the trained network and the numerical ground truth.

Test case	Relative error
Nominal (E_0, q_0)	0.50%
Softer material ($0.8 E_0$)	3.08%
Stiffer material ($1.2 E_0$)	0.66%
Higher load ($1.5 q_0$)	5.50%
Extreme load ($2.0 q_0$)	13.87%

Error is smallest near the training distribution’s center and grows further out, as expected. The last row is a deliberate extrapolation test: since the load’s uncertainty distribution has only 15% coefficient of variation, a load twice the nominal value is far outside anything the network saw during training, and the visible divergence in Figure 3 reflects that. The Monte Carlo reliability study below only samples within the trained distribution, where accuracy is much higher.

5.3 Reliability analysis

Figure 4 and Table 3 show the results of the 20,000-sample Monte Carlo study.

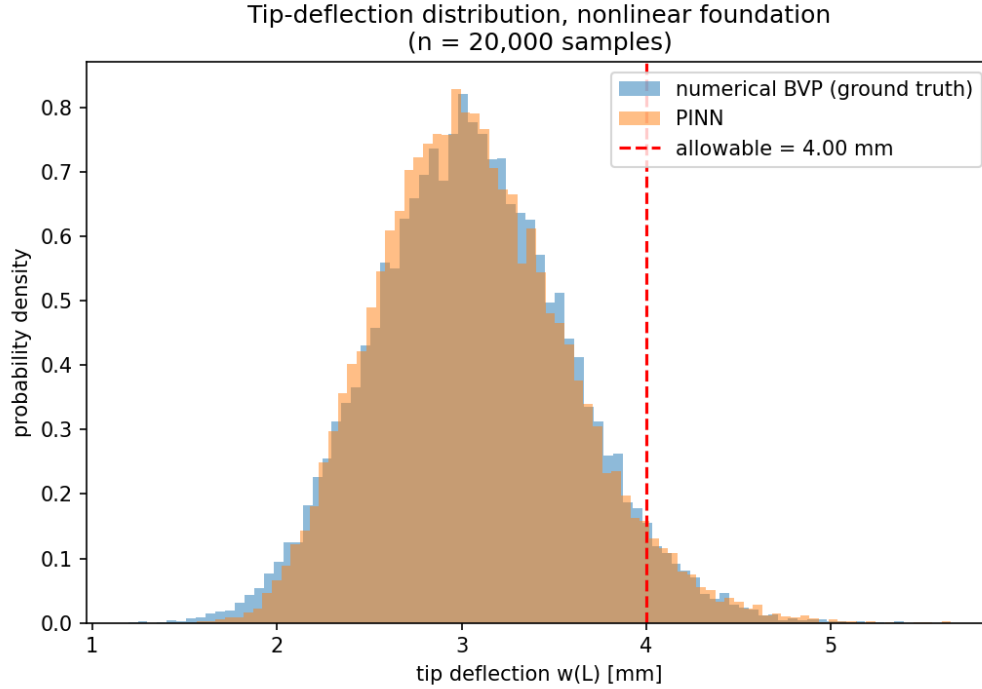


Figure 4: Distribution of tip deflection over 20 000 random material/load combinations, from the trained network (orange) and the numerical BVP ground truth (blue).

Table 3: Reliability statistics over 20,000 Monte Carlo samples of (E, q) .

Quantity	Trained network	Numerical baseline (ground truth)
Mean tip deflection	3.065 mm	3.064 mm
Standard deviation	0.515 mm	0.527 mm
5th percentile	2.285 mm	2.226 mm
95th percentile	3.972 mm	3.958 mm
Probability of exceeding 4.00 mm	4.61%	4.32%

The overall relative error between the network’s full set of 20 000 predicted deflections and the numerical ground truth is **1.51%**, and all 20 000 numerical BVP solves converged successfully.

5.4 Timing comparison

Table 4 and Figure 5 report how long each method took to evaluate the full 20 000-sample Monte Carlo study, and how that cost scales with the number of samples.

Table 4: Wall-clock time to evaluate 20,000 material/load scenarios.

Method	Time for 20,000 evaluations
Numerical BVP solver (one solve per sample, in a loop)	24.87 s
Trained PINN (one batched forward pass)	5.4 ms
Speed-up factor	$\approx 4,640\times$

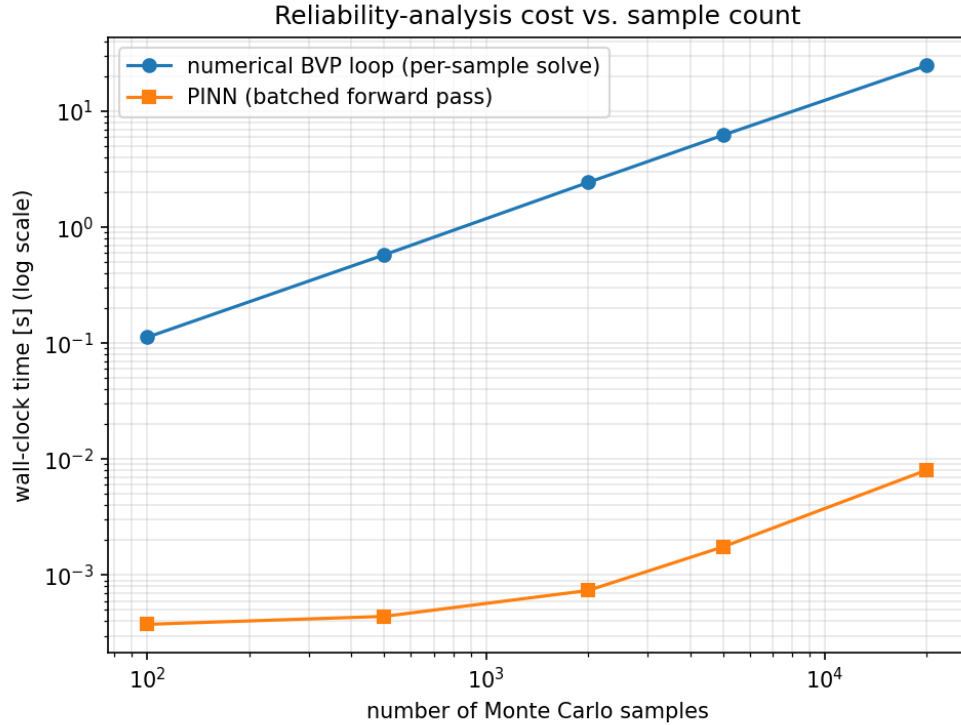


Figure 5: Wall-clock time to evaluate a Monte Carlo reliability study, as a function of the number of samples, for both methods (log–log axes).

The numerical baseline’s cost grows roughly linearly with the number of samples, since each sample requires an independent solve. The trained network’s cost stays close to flat and orders of magnitude lower across the entire tested range, from 100 to 20,000 samples. At the full scale used in this study, this is the difference between a study that finishes in under one hundredth of a second and one that takes roughly 25 seconds.

6 Discussion

The timing gap in Section 5.4 reflects a difference in how cost scales, not just a one-off speed difference. A numerical solver must redo its iterative solution process from scratch for every new scenario, so its total cost grows with the number of scenarios examined. A trained network pays its entire solving cost once, during training; every subsequent evaluation is a fixed, small amount of arithmetic, regardless of how many scenarios are needed. That difference in cost structure is what

produces the widening separation in Figure 5, and it would grow further, not shrink, on a more expensive underlying problem such as a full finite-element model of a real structure.

The 13.87% error at double the nominal load (Table 2) shows that the network should only be trusted within, or close to, the range of conditions it was trained on. The Monte Carlo study only sampled from the trained distributions, and its resulting failure probability (4.61%) matched the numerical ground truth (4.32%) closely as a result.

7 Where the Speed-Up Actually Comes From

The timing comparison in Section 5.4 raises a natural question: is the trained network faster because it does *less arithmetic* per sample, or for some other reason? This matters, because if the answer were “less arithmetic,” one might reasonably ask whether a few percent of extra error is a fair price for a small computational saving. We checked this directly by counting the operations on both sides.

7.1 Counting the operations on each side

The trained network’s forward pass is a fixed, exactly countable computation: 4 hidden layers of 40 units each, taking 3 inputs and producing 1 output, gives 5121 trainable parameters and **4,960 multiply-add operations** (about **9,920 floating-point operations**) every time it is evaluated, plus 160 evaluations of the tanh nonlinearity. This cost is exactly the same for every input — it does not depend on E or q — and is fully parallelizable: all 20 000 Monte Carlo samples are evaluated together as a single batched matrix multiplication.

The numerical baseline’s cost is harder to pin down in advance, since it depends on how many iterations the solver needs, but it can be measured directly. Instrumenting `solve_bvp` with call counters over 300 random (E, q) samples gives, on average: about **26 calls** to the physics right-hand-side function, evaluated across roughly 30 mesh points each time (about **769 total point evaluations**), plus about **16** boundary-condition evaluations, all within a single mesh-refinement pass (the initial 30-point mesh already met the requested tolerance in every sampled case). Each point evaluation of the physics formula $(q - k_1 w - k_3 w^3)/(EI)$ costs only about 7 elementary floating-point operations, so the raw physics arithmetic performed per sample is roughly $769 \times 7 \approx$ **5,400 FLOPs**.

Table 5: Raw arithmetic cost per sample: trained network vs. numerical solver.

	Trained network (forward pass)	Numerical baseline (one BVP solve)
Elementary floating-point operations	≈ 9920	≈ 5400
Depends on the specific (E, q) ?	no — fixed cost	yes — iteration count varies
Batchable across many samples?	yes, trivially	no — one independent solve each

The two raw-arithmetic figures are the *same order of magnitude*; the network does not win by doing meaningfully less floating-point math per sample. The speed-up demonstrated in this report

is not a result of doing fewer floating-point operations.

7.2 Where the numerical solver’s time actually goes

Profiling 500 individual `solve_bvp` calls (Python’s `cProfile`) shows that the physics right-hand-side function itself — the actual arithmetic content of the problem — accounts for only about 20% of the wall-clock time, and much of that 20% is Python and NumPy call overhead (function-call dispatch, array construction) rather than the 7 FLOPs of real math being computed at each point. The dominant cost, at roughly 68% of the total time, is the **Newton iteration machinery**: numerically estimating a Jacobian matrix by finite differences, assembling a sparse matrix representing the linearized collocation system, and factorizing and solving that sparse linear system. This is standard, unavoidable overhead for any general-purpose nonlinear boundary-value-problem solver, and it must be repeated, from scratch, for every single (E, q) combination.

7.3 The real mechanism: parallel batching, not fewer computations

Putting these two findings together gives a more precise explanation of the speed-up than “the network does less work”:

- The *amount* of raw arithmetic needed to represent the physics is comparable on both sides — a few thousand floating-point operations either way.
- The numerical solver spends most of its time on *iterative, serial, per-sample* machinery (Jacobian estimation, sparse factorization, dozens of separate Python-level function calls) that cannot be shared or reused across different (E, q) samples.
- The trained network reduces the entire problem to a single, fixed-shape matrix multiplication, which modern numerical libraries execute as one highly parallel operation across all 20 000 samples simultaneously, with none of the per-sample iteration overhead.

In short: the speed-up comes from trading a serial, iterative, per-sample procedure for a single parallel, batched computation — not from reducing how much arithmetic the underlying physics requires.

8 Is the Accuracy Trade-off Worth It?

Framed this way, the right question is not “is a few thousand FLOPs of savings worth a few percent of error” — there is essentially no FLOP saving to weigh against the error in the first place. The real question is whether trading a small amount of accuracy for the ability to batch across many samples in parallel is worth it, and that depends entirely on how the tool would actually be used.

- **A single reliability study on one beam design.** Here, the numerical baseline already finishes in about 25 seconds (Table 4). There is no real waiting problem to solve, so accepting any accuracy cost is hard to justify in this narrow case.
- **A design-optimization loop that repeats the reliability study many times.** If an outer loop needs to evaluate, say, a thousand candidate beam designs, each requiring its own 20 000-sample Monte Carlo pass, the numerical route would take on the order of seven hours in total, while the trained-network route would still take seconds. This is the regime where the batching advantage compounds into something that matters.
- **A more expensive per-sample solver.** On a real 2D or 3D finite-element model, a single solve can take minutes rather than milliseconds. A full Monte Carlo campaign in that setting

may not be numerically feasible *at all* within a practical time budget; there, an approximate-but-obtainable answer can be more useful than an exact-but-unobtainable one.

- **A hybrid, lower-risk option.** The trained network can be used only as a cheap screening tool, to identify which (E, q) region sits close to the failure boundary, with the certified numerical solver then re-verifying just those critical points. This captures most of the speed advantage while keeping the final, reported answer numerically exact.

9 Improving Accuracy: Training on a Wider Distribution

The largest source of error identified in Section 5 was extrapolation: the network was noticeably less accurate at the extreme-load test case (double the nominal load, Table 2), which sits far outside the 15% coefficient-of-variation load distribution used for training. We tested a direct fix: retrain the network on a *wider* range of (E, q) than the true physical uncertainty distribution, so that the tail scenarios are no longer out-of-distribution extrapolation for the network, while leaving the *true* distribution used for the actual Monte Carlo reliability study unchanged. Table 6 summarizes the changes made.

Table 6: Changes made for the wider-distribution training run.

Setting	Original run	Wider-distribution run
Load CoV used for <i>training</i> sampling	15% (= true CoV)	35%
Young’s-modulus CoV used for <i>training</i> sampling	10% (= true CoV)	18%
Load/modulus CoV used for the <i>Monte Carlo study</i>	15% / 10%	15% / 10% (unchanged)
Collocation points per batch	1200	1500
Boundary points per batch (each end)	150	200
Hidden-layer width	40	48
Adam epochs	3500	4500
L-BFGS iterations	400	500

9.1 Curve-fitting accuracy improved substantially

Table 7: Relative L2 error against the numerical ground truth: original vs. wider-distribution training.

Test case	Original	Wider-distribution training
Nominal (E_0, q_0)	0.50%	0.89%
Softer material ($0.8 E_0$)	3.08%	0.33%
Stiffer material ($1.2 E_0$)	0.66%	1.49%
Higher load ($1.5 q_0$)	5.50%	1.30%
Extreme load ($2.0 q_0$)	13.87%	4.48%
Full 20,000-sample Monte Carlo relative error	1.51%	1.20%

Figure 6 shows the same visual comparison as Figure 3, using the wider-distribution model: the extreme-load case (cyan dashed line) now tracks its numerical ground truth (olive solid line) far more closely.

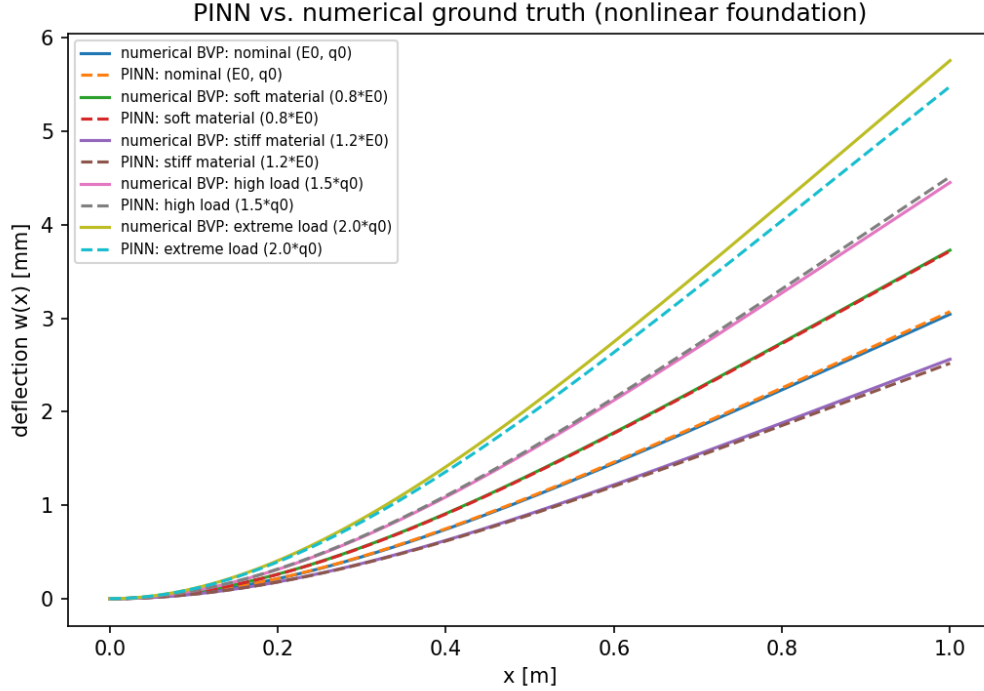


Figure 6: Predicted deflection curves after training on a wider (E, q) distribution. Compare the extreme-load case (cyan dashed vs. olive solid) to the same case in Figure 3.

The extreme-load error dropped by roughly a factor of three ($13.87\% \rightarrow 4.48\%$), and the higher-load case dropped by roughly a factor of four ($5.50\% \rightarrow 1.30\%$), at the cost of a small increase in error at the nominal and stiffer-material cases (both still under 1.5%). The network’s overall accuracy across the full Monte Carlo sample set also improved slightly ($1.51\% \rightarrow 1.20\%$).

9.2 The reliability estimate did not improve

Table 8 shows the resulting reliability statistics.

Table 8: Reliability statistics: original vs. wider-distribution training (true Monte Carlo sampling distribution unchanged in both cases).

Quantity	Original PINN	Wider-distribution PINN	Numerical base-line
Mean tip deflection	3.065 mm	3.081 mm	3.064 mm
Standard deviation	0.515 mm	0.549 mm	0.527 mm
P(exceed 4.00 mm)	4.61%	5.09%	4.32%
Gap from the numerical baseline	0.29 pp	0.77 pp (worse)	—

Despite fitting the deflection curves more accurately almost everywhere, the wider-distribution network’s estimate of the failure probability moved slightly *further* from the numerical ground truth, not closer. The reason is that both the mean and the standard deviation of its predicted deflection distribution shifted slightly upward, and because the allowable limit (4.00 mm) sits close to the peak of the deflection density (Figure 4), even a small systematic shift there moves a disproportionate amount of probability mass across the threshold. So a model that fits better on average is not necessarily more accurate on the specific quantity that matters most. Curve-fitting error and tail-probability error are different things to check, and checking only the former can give a wrong impression of accuracy where it counts.

9.3 Directions more likely to fix the reliability estimate specifically

Rather than continuing to widen the training distribution generally, the following are more directly targeted at the failure-probability estimate itself:

1. **Oversample collocation points near the failure boundary.** Instead of sampling q uniformly across its training range, concentrate additional training points at load values that push the deflection close to w_{allow} , since that is precisely the region the reliability estimate is sensitive to.
2. **Post-hoc bias correction.** A cheap, no-retraining fix: use the small measured mean/standard-deviation offset from one numerical Monte Carlo run to rescale the network’s output distribution before computing the failure probability.
3. **Train longer on the wider distribution.** The wider-distribution run’s loss was still decreasing when training stopped (Table 6 increased epochs only modestly); additional epochs may close some of the remaining bias on their own.
4. **Adaptive, residual-based resampling.** Rather than sampling training points uniformly at random, periodically add extra collocation points where the current physics residual is largest — a standard PINN refinement technique that tends to be more sample-efficient than uniformly increasing the number of training points.

10 Further Possibilities

1. **Move to a problem where the numerical baseline itself is expensive.** Replacing the 1D `solve_bvp` baseline with a full finite-element simulation of a 2D or 3D structure, where a single solve might take seconds to minutes, would likely widen the speed-up by additional orders of magnitude, since the network’s per-scenario cost stays essentially fixed regardless of problem complexity.
2. **Parallelize the numerical baseline.** Distributing the 20,000 individual solves across multiple CPU cores would reduce the 25-second figure, but would not change the underlying scaling: numerical cost would still grow with the number of samples, while the network’s would not.
3. **Use the trained network as a base for more advanced reliability techniques.** Because evaluating the network is so cheap, techniques such as importance sampling or adaptive sampling near the failure boundary become practical to run directly on top of the trained surrogate.
4. **Explore other sources of nonlinearity.** Large deflections (geometric nonlinearity), nonlinear material behavior such as plasticity, and contact problems are other natural candidates

for structural problems with no closed-form solution.

11 Conclusion

A cantilever beam on a nonlinear elastic foundation has no closed-form solution, so every reference answer requires a numerical boundary-value-problem solve. Training a physics-informed neural network on this problem produced a model that matches the numerical ground truth closely near typical operating conditions, degrades predictably outside the range it was trained on, and delivers a full 20,000-sample Monte Carlo reliability estimate roughly **4,600 times faster** than solving the same 20,000 scenarios numerically, while agreeing with the numerical estimate of the probability of failure to within half a percentage point.

That speed-up comes from a difference in computational *structure*, not from the network performing less arithmetic: the raw floating-point cost of a forward pass and of one numerical solve are comparable, but the numerical solver’s cost is serial and per-sample while the network’s is a single operation that batches across every sample at once (Section 7). Whether that trade is worthwhile depends on how the tool is used (Section 8). A follow-up experiment training on a wider (E, q) range showed that improving average curve-fitting accuracy does not automatically improve the specific reliability number of interest, since that number is disproportionately sensitive to small biases near the allowable-deflection threshold (Section 9). The accuracy metric to check is the one tied to the actual decision being made, not just overall curve error.

The problem setup, the network size, and the fixes tried in Section 9 were our choices among several reasonable options, not the only way to approach this. The full code is at <https://github.com/officialnamit/Physics-Informed-Neural-Networks> so that any of these choices — the foundation model, the network architecture, the training distribution, the Monte Carlo setup — can be checked, changed, or extended directly.

A Summary of Notation

Symbol	Meaning
x	position along the beam, $0 \leq x \leq L$
$w(x)$	transverse deflection of the beam at position x
E	Young’s modulus (material stiffness)
I	second moment of area (cross-section shape factor)
q	uniformly distributed transverse load
L	beam length
k_1	linear elastic foundation modulus
k_3	cubic (hardening) foundation modulus
w_{allow}	allowable (serviceability) tip deflection
y_1, y_2, y_3, y_4	first-order state variables (w, w', w'', w''') used in the BVP formulation